2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS) | 979-8-3503-3766-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/IPDPS54959.2023.00050

Chic-sched: a HPC Placement-Group Scheduler on Hierarchical Topologies with Constraints

Laurent Schares, Asser Tantawi, Pavlos Maniotis, Ming-Hung Chen,
Claudia Misale, Seetharami Seelam, Hao Yu
IBM Thomas J. Watson Research Center, Yorktown Heights, NY, USA
schares@us.ibm.com, tantawi@us.ibm.com, ppmaniotis@ibm.com, minghungchen@ibm.com,
c.misale@ibm.com, sseelam@us.ibm.com, yuh@us.ibm.com

***Abstract*—Efficient placement of advanced HPC and AI workloads with application constraints is raising challenges for resource schedulers on shared infrastructures, such as the Cloud. In this work, we propose a novel Constraints- and Heuristics-based scheduler on HIerarchical Topologies for High-Performance Computing workloads in the Cloud (*chic-sched*, for short). Our heuristics-based algorithm enables placement across multiple levels in a network hierarchy with loosely specified constraints, and it works without retries by providing suboptimal placements to minimize placement failures. This allows for fast scheduling at scale, and the $O(N \log N)$ complexity enables placement decisions within tens of milliseconds for groups of hundreds of virtual machines (VM). We introduce a new and simple metric to quantify the goodness of group placements. With this metric, in terms of deviation from ideal placements, we show that *chic-sched* is 20-50% better than the common bestFit or worstFit algorithms in all scenarios of two-level placements with spreading and packing constraints. We evaluate *chic-sched* with publicly available VM-request traces from a production Cloud, and, comparing against bestFit, we show that it achieves 8% lower placement failure rates and more than 40% better placement locality. Finally, to quantify the goodness of constraints-based placements, we conduct experiments with a realistic MPI workload on synthetically allocated VM clusters in a public cloud. We measure a 9% performance improvement over an adverse placement in a scenario where our heuristics-based scheduler would return a good, but not perfect, placement.**

***Keywords*—Scheduling, Placement Groups, HPC in Cloud**

I. Introduction

A significant part of the High-Performance Computing (HPC) market is migrating to the Cloud, as the on-demand nature of running HPC in the Cloud may lead to total-cost-of-ownership improvements over dedicated HPC while delivering similar or equal application performance [1]. This is driven by the availability of high-performance computing and networking infrastructure by Cloud service providers, the possibility of cloud bursting for elastic workflows, and the choice among specialized compute instances running on the latest hardware. A similar trend is emerging in Artificial Intelligence (AI), as a cloud-native approach offers better integration potential across the software stack for both training and inference compared to dedicated AI clusters [2], [3].

However, since many applications run concurrently on a shared infrastructure in the Cloud, distributed workloads are subject to performance degradation if their tasks are not efficiently placed. Performance impacts can be on the order of 10% to over 2x, and they include network-bound effects-—such as bandwidth oversubscription, zero-load and tail latencies, or slowdowns/failures in a storage subsystem-—and compute-bound effects-—such as noisy-neighbor VMs on a compute node competing for shared resources like memory bandwidth, or processes stealing compute cycles and causing delays in message completion. For this reason, cloud providers make *placement groups* available to users requesting a set of resources for their workloads. Since cloud users have no visibility to the physical infrastructure, placement groups help capture user preferences and translate them to physical locations. Simple examples include *affinity (packing)*, often used for tightly-coupled applications that need low and bounded network latency, or *anti-affinity (spreading)* for high-availability scenarios that cannot afford any downtime, and these are commercially available to varying degrees in major Clouds today [4]–[9].

Placement groups can be far more involved, and they can combine affinity and anti-affinity at different levels in a network hierarchy, such as packing within a compute rack while spreading at the server-level. Or they can add *constraints* at each level: spreading but not too thin, or packing but not too densely, or the degree of enforcement (strict or best-effort). Examples include workloads that are simultaneously bound by memory- and network-bandwidth, such as AI and Foundation Model training, or various HPC/MPI patterns: these applications should be (1) *packed* at one level of the network hierarchy (e.g. a rack) to minimize network congestion effects, while being (2) *spread* equally across servers within the rack to ensure that each VM has the same memory bandwidth. A different set of constraints is beneficial for another class of applications, such as data stores and distributed databases: (1) *packing* at the node-level for high memory bandwidth and energy reduction, while (2) *spreading* the nodes across racks to minimize north-south network congestion for applications that need to provide high bandwidth to their clients, and (3) *spreading* across availability zones for reliability. Other types of constraints can be related to resource disparities or to restrictions caused by software licensing terms. These factors dictate how to schedule on servers with different types of processors, on racks with power-supply limits, or within parts of the infrastructure providing enhanced network performance. Placement considerations are not limited to the

1530-2075/23/$31.00 ©2023 IEEE
DOI 10.1109/IPDPS54959.2023.00050

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

Cloud, and constraints are equally, if not more, stringent in Edge-Computing or Enterprise data centers (DCs) that are often subject to strict power-supply, cooling, bandwidth- or regulatory compliance conditions.

While placement constraints provide many benefits, there are practical limitations. First, it can be hard or impossible for a cloud provider to provide a set of physical resources that fully satisfies the constraints, which may lead to failure of allocating a requested job. And second, the scheduling complexity of using many constraints may lead to slow placement times at scale. In this work, we address these concerns by designing a heuristics-based algorithm for fast scheduling without retries. While this may result in suboptimal allocations, cloud users or administrators posting the resource allocation request are often willing to accept these if they are "good enough".

The key novelty of our work is twofold: first, the combination of efficient heuristics-based placement-group scheduling with loosely specified constraints at multiple levels of a network hierarchy, and second, a new metric to quantify the goodness of placement decisions. The placement solution is a logical topology that takes the constraints into account, and the metric enables the requester to accept or reject the placement and to decide on how to best run applications on non-optimally scheduled resources.

This paper is structured as follows. We introduce our design guidelines and describe the problem in Sec. II.The placement algorithm is presented in Sec. III, and examples of using the algorithm with multi-level constraints are provided in Sec. IV. Sec. V covers our measure of goodness for the quality of placement. A detailed evaluation is presented next: trace-driven simulations are provided in Sec. VI, an experiment using placements with constraints is in Sec. VII, and a scalability analysis is covered in Sec. VIII. Section IX covers related work, and we conclude in Sec. X.

II. Design Guidelines and Problem Description

In this section, we present our design guidelines and formulate the problem statement. Our main criteria include: consistent network performance with bounded latency at scale, operational simplicity, low complexity for fast scheduling times, minimal resource fragmentation, high utilization, and minimizing placement failures. Workload locality also matters, as it decreases the likelihood of network congestion for all tenants. While topology can be represented as general graphs at the network-, node-, or NUMA-levels, we focus on tree topologies in this paper. A schematic view of placement group scheduling is shown in Fig. 1. We design our scheduling algorithm as a plugin to an existing resource management framework. It supports a range of placement constraints, including placing tightly-coupled applications within the lowest-possible level of the network hierarchy, partitioning applications into multiple groups, and more complex cases such as spreading at one level while packing at another one.

Given a set of heterogeneous physical entities, $\mathcal{N}$, e.g. physical machines (PM), each with some resource capacities, and a hierarchical topology represented by a physical tree,

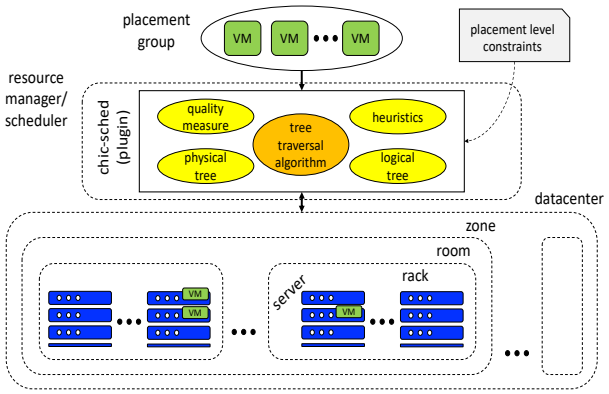


Fig. 1: Schematic view of placement group scheduling on a hierarchical topology

$pTree$, with $\mathcal{N}$ as leaves in the tree, the problem is to place a set of homogeneous logical entities, $\mathcal{M}$, e.g. virtual machines (VM), each with some resource demands, onto $\mathcal{N}$ such that (1) the resource demands are satisfied by the available capacities and (2) a set of level constraints, $\mathcal{C}$, e.g. pack the VMs on racks of PMs, are satisfied. Examples of a physical entity are: a server or a NUMA node in a DC or cloud infrastructure, a VM or node in a cluster of a container cloud platform. Examples of a logical entity are: a VM in a virtualized environment, a container (or pod of containers) in a containerized environment. Examples of resources are: CPUs, memory, storage, accelerators, GPUs, and network links.

The physical tree is a containment hierarchical topology where the levels, from the leaves to the root, may represent servers, racks, rows or rooms of racks, availability zones, and DC regions. $pTree$ is a full tree with L levels, where leaf nodes are at level 0 and the root node is at level L. A physical node is denoted by $pNode$. Level constraints $\mathcal{C}$ is a set of constraints, each at a given level l, $l = 0, 1, \cdots, L$, specifying how multiple logical entities are to be placed on physical entities at level l. Constraints determine whether a placement uses, for example, a $Pack$ or $Spread$ strategy, and whether the constraint is soft (best effort) or hard.

Resources are assumed to be consumable, i.e. a resource demand vector, $\mathbf{D}$, is satisfied by a resource available capacity vector, $\mathbf{A}$, if $\mathbf{D} \leq \mathbf{A}$, where the elements of a vector correspond to resource types. For convenience, when placing logical entities $\mathcal{M}$, we define a $slot$ to be a set of resources on a physical entity needed to satisfy the resource demand of a single member of $\mathcal{M}$.

A placement group $\mathbf{G}$ consists of the members $\mathcal{M}$ along with the constraints $\mathcal{C}$, i.e. $\mathbf{G} = \{\mathcal{M}, \mathcal{C}\}$. A placement is an assignment of each of the members of $\mathcal{M}$ to a member of $\mathcal{N}$, satisfying $\mathcal{C}$ and resource demands. If a solution to this problem is not found, then the placement operation either fails and all members are to be placed as a group, or a partial placement is returned.

A placement solution is represented by a tree, called logical tree $lTree$, which is a (nonempty) subtree of $pTree$, where

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

the root of $lTree$ maps to the root of $pTree$ and the leaves of $lTree$ map to a (nonempty) subset of the leaves of $pTree$. A logical node is denoted by $lNode$ and maps to a $pNode$ at the same level. A $lNode$ has a count of the members of $\mathbf{G}$ placed at its mapped $pNode$. A successful (complete) placement results in a $lTree$ with a count of the root equal to the group size, $M = |\mathcal{M}|$.

In this paper, we consider primarily two types of level constraints: $Pack$ and $Spread$ (extensions are discussed in section III-E). We provide a heuristics-based algorithm to solve the placement problem described above; this algorithm traverses the $pTree$ in depth-first order once, until either a solution is found or it fails to find a solution. For convenience, table IV in appendix A lists the main symbols in the paper.

III. Algorithm

First, we describe the main attributes of the $pNode$, $lNode$, level constraint, and group entities in table I.

A. Highlights of the algorithm

An outline of the heuristics-based placement algorithm follows. Starting from the root of the $pTree$ and traversing the $pNodes$ in a depth-first order, we solve a placement problem at each visited node, n, the outcome of which is a subset of $\mathcal{M}$ being placed on n and an $lNode$, n', associated with n, is created, until all the set $\mathcal{M}$ is placed and an $lTree$ is generated. Two major issues that need to be resolved in this outline are: (1) the order in which the children of n is visited, and (2) the determination of the number of members of $\mathcal{M}$ to be attempted to place on n.

1) Ordering of nodes: Assume that the number of slots available at the subtree rooted at $pNode$, n, denoted by $A(n)$, is calculated for all nodes in the $pTree$. This is done initially starting from the leaves and up to the root. For a leaf node n, $A(n)$ is simply obtained by dividing the available resources on the node by the resource demand of a member of $\mathcal{M}$. For an internal node n, $A(n)$ is the sum of $A(\nu)$ over its children, $\nu \in children(n)$.

TABLE I: Description of attributes

attribute	description
pNode	
children	list of children of $pNodes$, empty if leaf node
level	the level of the $pNode$ in the $pTree$
capacity	a vector of resource capacities
allocated	a vector of resource allocations
slots	number of available slots to accommodate members of group $\mathcal{M}$
lNode	
pNode	reference to the $pNode$ this $lNode$ is mapped to
count	the number of members in group $\mathcal{M}$ placed in the subtree rooted at this $lNode$
Constraint	(abbreviated $const$)
level	the level at which this constraint is applied
type	$Pack$ or $Spread$
isHard	true (hard) or false (soft)
Group	
size	the number of members in group $\mathcal{M}$
demand	the resource demand vector for a member in the group
constraints	a list of level constraints
lTree	the $lTree$ representing the placement for the group

TABLE II: Placement range

Constraint	Pack	Spread
Hard	$[M(n), M(n), M(n)]$	$[1, 1, 1]$
Soft	$[1, M(n), M(n)]$	$[1, \lceil M(n)/N(n) \rceil, M(n)]$

The children of node n are then ordered according to the values of their $A(\nu)$. In particular, the children are sorted in decreasing order if the type of constraint at the level of the children is $Pack$, and in increasing order if it is $Spread$ [1]. The reasoning is that, in the case of $Pack$, we want to fit as many members as we can in the subtree that has the most availability. And, in the case of $Spread$, we want to fit the least minimum first, then attempt to maintain at least that number in the remaining subtrees.

2) Determination of number to place: As for the attempted number of group members to place at node n, we define a range, $R(n)$, as the tuple $\{a, d, b\}$, where a and b are minimum and maximum values, respectively, and d is a desired value within the interval $[a, b]$. When visiting node n, let $M(n)$ be the number of group members to attempt to place on n. Further, let $N(n)$ be the number of sibling nodes of n that are yet to be visited, including node n itself. Then, the range $R(n)$ is determined based on the type and hardness of the constraint at the level of node n, as given in table II. The case of $Hard$ constraint is straightforward: either all or exactly one of the remaining members are to be placed, for $Pack$ and $Spread$ types, respectively. For $Soft$ constraint, the minimum is 1 and the maximum is $M(n)$ for both types. The desired value, however, depends on the type: it is all of the remaining in the case of $Pack$ and means equally spreading the remaining members among the remaining nodes in the case of $Spread$.

Let $Z^*(n)$ be the best choice for the number of members to place at node n. Then, given $R(n)$, it is evaluated as,

$$Z^*(n) = \begin{cases} min\{min\{A(n), b\}, d\}, & A(n) \geq a \\ 0, & A(n) < a, \end{cases} \quad (1)$$

resulting in a value as close as possible to the desired one within the given range. The actual number placed at node n, after visiting all nodes in its subtree, is denoted by $Z(n) \leq Z^*(n)$.

At node n, given the number of group members to attempt to place, $M(n)$, and the number of remaining siblings (i.e. with same parent), $N(n)$, including node n, a placement range, $R(n)$, is calculated as described in table II, and a best choice for the number of members to place, $Z^*(n)$, is calculated using the number of available slots, $A(n)$, as described in equation 1, then the actual number placed, $Z(n)$, is returned. The latter is obtained after visiting the children of node n, ordered as described above, accumulating the $Z(\nu)$ returned by the children, $\nu \in children(n)$. Visiting children nodes

[1] In case there is no constraint specified at the given level, then a default type of $Pack$ is set, as this leads to a more efficient bin packing for heterogeneous logical entities.

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

stops whenever $Z(n) = Z^*(n)$ is satisfied, or the set of children is exhausted. In the case of n being a leaf node, then $Z(n) = min\{A(n), Z^*(n)\}$, bounded by the number of available slots.

The above procedure starts from the $root$ of the $pTree$. The returned value $Z(root)$ is the total number placed. If it is equal to the group size, then the group succeeded a holistic placement, otherwise the group is only partially placed. As $pNode$, n, is visited, the associated $lNode$, n', is created and its count is set to $Z(n)$. Node $root'$ becomes the root of the $lTree$, representing the logical placement for the group.

B. Group placement algorithm

The group placement algorithm is summarized in algorithm 1. The group is abbreviated by g, its size by $g.size$, its set of constraints by $g.const()$, and a particular constraint at level l by $g.const(l)$. In case constraint $g.const(l)$ is not specified, the default one (Pack, Soft) is used instead.

Algorithm 1 Logical assignment of placement group

```
1:  procedure PLACEGROUP(pTree, g) return lTree
2:      for all n ∈ pTree do
3:          calculate A(n)                          ▷ section III-A1
4:      M(pTree.root) ← g.size
5:      N(pTree.root) ← 1
6:      root' ← PLACEATNODE(pTree.root)
7:      return newLTree(root')

8:  procedure PLACEATNODE(n) return lNode
9:      n' ← newLNode(n)
10:     l ← n.level
11:     R(n) ← RANGE(n, g.const(l))
12:     Z*(n) ← BEST(R(n), A(n))                    ▷ equation 1
13:     if Z*(n) = 0 then return n'                 ▷ skip n
14:     if l = 0 then                               ▷ n leaf node
15:         Z(n) ← min{A(n), Z*(n)}
16:     else
17:         C ← ORDERNODES(n.children, g.const(l − 1))
18:         Z(n) ← 0; siblings ← |C|
19:         for all c ∈ C ∧ Z(n) < Z*(n) do
20:             M(c) ← Z*(n) − Z(n)
21:             N(c) ← siblings
22:             c' ← PLACEATNODE(c)
23:             if Z(c) > 0 then
24:                 Z(n) ← Z(n) + Z(c)
25:                 n'.addChild(c')
26:             siblings−−
27:     n'.count ← Z(n)
28:     return n'
```

C. Complexity

Let N_d be the total number of nodes in the $pTree$, N the number of leaf nodes, i.e. physical entities, and M the group size. Since each node is visited at most once, and the number of operations per node is $O(1)$, then the complexity is $O(N_d)$, except for the step of node ordering. For c children, the complexity of node ordering is $O(c \log c)$. Thus, the worst case is a one-level tree where the number of children of the root node is N, resulting in a complexity of $O(N \log N)$.

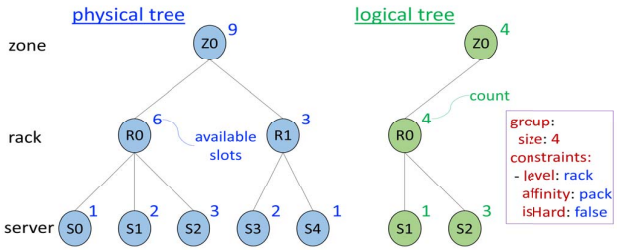


Fig. 2: Example: group placement

D. Illustrative Example

As an illustration, Fig. 2 presents a small example of a physical tree representing a DC with a single zone Z_0, two racks: R_0 and R_1, three servers on the first rack: S_0, S_1, and S_2, and two servers on the second rack: S_3 and S_4.

For simplicity, we consider a set of homogeneous servers, each having two resources: CPU (number of cores) and memory (in GB), with capacities $[16, 256]$, respectively. We need to place a group of four homogeneous VMs, each with demand $[4, 32]$ and a single level constraint at the rack level with soft pack type. (If unspecified, the default constraint at a level is soft pack.) Some resources are already allocated at the various servers, as shown below. Accordingly, the number of available slots for members of the group are calculated and presented in Fig. 2.

The resulting placement is shown in the form of a logical tree, where all four VMs are allocated on rack R_0, with three VMs on server S_2 and one VM on server S_1. Based on slot availability and the pack constraint, nodes R_0 and R_1 were ordered as $\{R_0, R_1\}$, thus rack R_0 was explored first. Further, the servers within rack R_0 were ordered as $\{S_2, S_1, S_0\}$. Accordingly, three of the four VMs in the group were placed on S_2 and the remaining one on S_1.

E. Extensions

The placement algorithm as described in algorithm 1 may be extended to incorporate additional features to the original group placement problem described in section II.

1) Range constraint per level: Consider a scenario where an HPC application needs a group of 128 VMs of a certain VM resource profile to be placed in the fewest racks possible, but uniformly distributed among the servers within a rack. For example, it is preferable to have 2 VMs on every server than a mixture of 1 and 3 VMs per server. This might be due to communication and synchronization patterns among the tasks of the application.

Thus, an additional constraint, $[n_{min}, n_{max}]$, at level l, $l = 1, 2, \cdots, L$, may be specified. This is only allowed if the placement constraint is soft at level l. Such a constraint is

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

easily handled by superimposing it on the values of a and b of the placement range, as defined in table II.

2) Group partitioning: For some applications, it is desirable to partition the placement group into a given number of partitions, where each partition is placed on a different unit at some level l, $l = 1, 2, \cdots, L$. An example would be dividing a group of 100 VMs into 5 partitions, where each partition is placed on a different rack (or zone or server). Furthermore, it is preferable that the partitions be of (almost) equal size. So, in the above example, each partition would have a size of around 20 VMs. Again, this is only allowed if the placement constraint is soft at level l.

Such a constraint is easily handled by superimposing the partition size on the desired value d of the placement range, as defined in table II. Also, the number of children nodes used to place instances of this group at level l would be limited to the specified number of partitions at l.

IV. Simulations of Placements with Constraints

In this section, we conduct simulation experiments and present the results of exercising the group placement algorithm presented in section III.

We consider a DC topology consisting of a single zone, 3 rooms in the zone, 8 racks per room, and 20 servers per rack, totaling 480 servers. This is representative of mid-sized enterprise or edge-computing DCs. For simplicity, we consider homogeneous servers, each having a single resource (CPU) with capacity of 16 cores. We simulate an initial state of an average 40% allocation per server. Due to the packing policy, mostly used in DCs to minimize fragmentation, we expect to have a portion of the servers completely empty and another one full. We assume probabilities of $1/8$ and $1/4$ for idle and full servers, respectively. The allocation on the remaining servers is assumed to have a coefficient of variation of 1.5, simulating the tendency of packing.

A group of size 120 VMs, each with a demand of 2 cores, needs to be placed in the DC, given the above-described initial state. We consider three different soft-level constraints: (a) pack on both racks and servers, (b) spread on racks and pack on servers, and (c) spread on both racks and servers. The placement results are shown in Fig. 3. For each, the plots in the left column show the allocation in rooms, racks, and servers, from top to bottom, respectively. The room placements are only shown once, in the top row, as they are identical for the three cases. The ordering on the x-axis is common among the three plots, showing the grouping. For instance, the first 20 servers make up the first rack; and the first 8 racks are in the first room. The y-axis shows the aggregate allocation. The shaded areas represent the background (initial) allocation before placing the group. The dark (colored) bars show the allocation due to the placement of the group of VMs. The plots in the right column depict the distribution of the number of VMs in the group that are allocated in the same room, rack, and server, going from the top to the bottom chart, respectively.

The first case of packing on racks is shown in Fig. 3 (a). All 120 VMs are placed in the same room, with 108 VMs

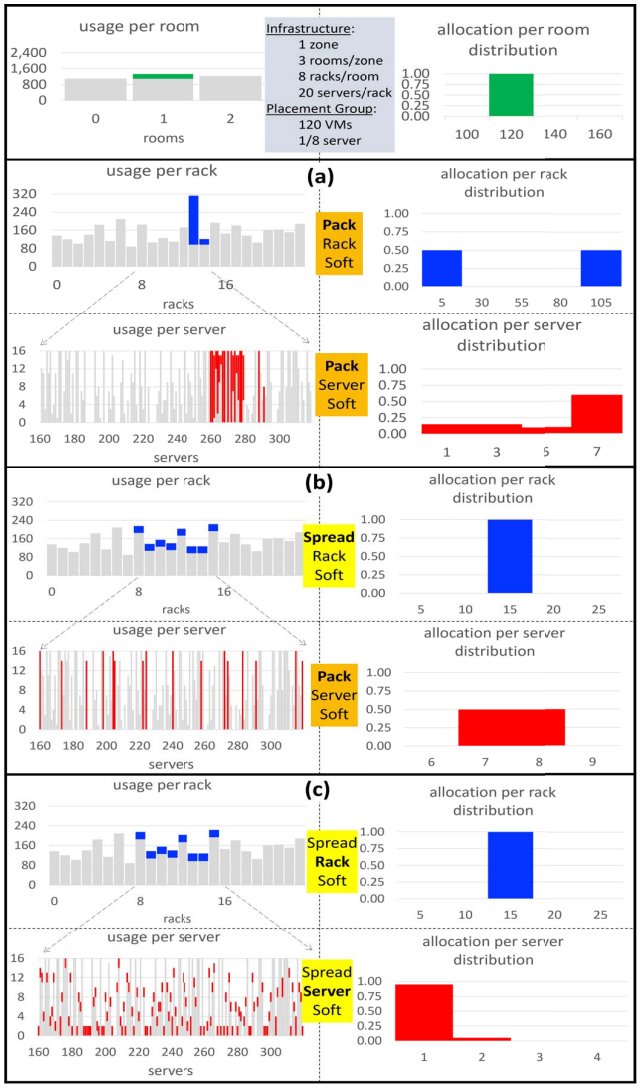


Fig. 3: Example: placement of groups with constraints.

placed on rack 13 and the remaining 12 VMs on rack 14. Within a rack, the VMs are packed (by default) on servers. The maximum number of VMs that fit on an empty server is eight. The distribution of the number of VMs placed on the same server demonstrates that most (60%) of allocated servers received 7 or 8 VMs, whereas the rest received almost uniformly anywhere between 1 and 6 VMs. More specifically, a total of 20 servers are used to place the group: 10 servers received 8 VMs each, 2 received 7 VMs, and the rest received anywhere between 1 and 5 VMs each. Note that, among the 8 racks (8 through 15) in room 1, the least-allocated rack 13 is picked and filled in to the most number of VMs (108) that could be placed on it. The remaining 12 VMs are spilled over to the next available rack 14.

The second case of spreading on racks while packing on servers is shown in Fig. 3 (b). All 120 VMs are placed in room 1, spread across all 8 racks (8 through 15) in the room,

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

with an evenly 15 VMs per rack, hence a perfect spreading. Within a rack, the 15 VMs are allocated on 2 servers, one receiving the maximum of 8 VMs and the other receiving the remaining 7 VMs.

Finally, the third case of spreading on both racks and servers is shown in Fig. 3 (c). As in the second case, all 120 VMs are placed in room 1, equally spread across all 8 racks in the room, each receiving 15 VMs. Within a rack, the 15 VMs are spread across the servers in the rack. A total of 108 servers received one VM each, and a few (6) servers received 2 VMs.

V. Measure of Goodness for Placement Quality

In this section, we introduce a measure of goodness for the quality of placement obtained by the algorithm with respect to the level constraint specified ($Pack$ or $Spread$). Typically, the impact of placement is measured by evaluating either application-level performance, such as throughput, response time, and networking, or bin-packing-related quantities, such as number of bins and bin usage [10]. Here, we are concerned with a more generic, application-independent measure for placing a group with multiple level constraints in a hierarchy.

A. Definition of measure

First, we define a measure for a given level with a normalized value in $[0, 1]$, where the extreme points 0 and 1 signify that the constraint is best and worst satisfied, respectively. As such, it measures the deviation from the objective. If the type of level constraint is $Pack$, then achieving a perfect $Pack$ results in a deviation of 0, and achieving a perfect $Spread$ results in a deviation of 1. The opposite is true if the type of level constraint is $Spread$.

The measure at a given level l, $0 \leq l < L$, is defined as follows. For simplicity, let us consider a particular physical entity at level $l+1$ which contains $m > 0$ physical entities at level l. Let $n > 0$ be the number of logical entities placed on those physical entities under a constraint at level l. For now, let us assume that all n logical entities can fit on one of the m physical entities and that $n \leq m^2$. Thus, a perfect $Pack$ results in all n logical entities placed on only one of the m physical entities, and a perfect $Spread$ results in each of the n logical entities placed on a separate physical entity. The placement algorithm may result in a placement pattern corresponding to either a perfect $Pack$ or $Spread$, or any other pattern in between. In particular, let the n logical entities be distributed among m' physical entities as $n_1, n_2, \cdots, n_{m'}$, such that $\sum_{i=1}^{m'} n_i = n$ and $n_i \geq n_{i+1}$, $i = 1, 2, \cdots, m'-1$, $1 \leq m' \leq m$. Given this ordering, denote the average number of physical entities weighted by the number of logical entities placed on them by a; it is given by

$$a = \frac{1}{n} \sum_{i=1}^{m'} i \, n_i, \tag{2}$$

[2]Relaxing these assumptions, we will have $m' > 1$ and $m' < n$ in the cases of perfect $Pack$ and $Spread$, respectively. Thus, we use equation 2 to calculate $a_{min} > 1$ and $a_{max} < (n+1)/2$, respectively. We omit the details here.

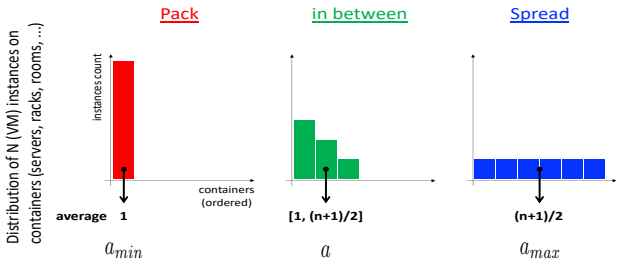


Fig. 4: Measure of deviation from Pack/Spread constraint.

$1 \leq m' \leq m$. In the case of perfect $Pack$, we have $m' = 1$ and $n_1 = n$, hence the minimum value of $a_{min} = 1$. And, in the case of perfect $Spread$, we have $m' = n$ and $n_i = 1, i = 1, 2, \cdots, m'$, hence the maximum value of $a_{max} = (n+1)/2$. An illustration is provided in Fig. 4.

Define $\tilde{a} \in [0, 1]$ to be the normalized value of $a \in [a_{min}, a_{max}]$ as [3]

$$\tilde{a} = \frac{a - a_{min}}{a_{max} - a_{min}},$$

$a_{min} < a_{max}$. For perfect $Pack$ and $Spread$ we get $\tilde{a} = 0$ and $\tilde{a} = 1$, respectively.

Next, we define a deviation measure, $\delta \in [0, 1]$, given a level constraint as

$$\delta = \begin{cases} \tilde{a}, & Pack, \\ 1 - \tilde{a}, & Spread, \end{cases}$$

$a_{min} < a_{max}$, otherwise we have, $\delta = 0$. A smaller deviation measure means that the placement algorithm achieved a good quality placement with respect to the given level constraint.

The above deviation measure δ was derived for a placement group, say g, at a given level, l, and a given size, n, at level $l+1$. Let us denote it by $\delta_g(l, n)$. Suppose that we have a total of C subgroups at level $l+1$ with sizes n_c, $c = 1, 2, \cdots, C$, then the deviation measure of g at level l is obtained by the weighted sum,

$$\delta_g(l) = \sum_{c=1}^{C} n_c . \delta_g(l, n_c) \; / \; \sum_{c=1}^{C} n_c$$

Let $\mathcal{L}$ be the set of levels where there are level constraints specified for group g. We define the overall group deviation factor, δ_g, as the distance deviation measure over $\mathcal{L}$, as

$$\delta_g = \sqrt{\frac{1}{|\mathcal{L}|} \sum_{l \in \mathcal{L}} \delta_g(l)^2}. \tag{3}$$

[3]An alternative expression is to use logs, as

$$\tilde{a} = (\log a - \log a_{min}) / (\log a_{max} - \log a_{min}),$$

with the reasoning of magnifying the sensitivity to the number of physical entities utilized when the number is small rather than large.

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

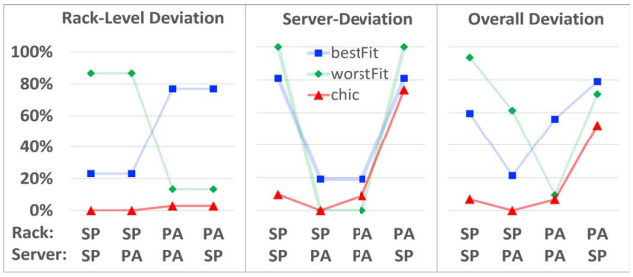


Fig. 5: Deviation measure comparison. Smaller is better. (PA=pack. SP=spread.)

B. Comparative evaluation

We conduct simulation experiments to evaluate the above deviation measure under our group placement algorithm and two standard bin-packing algorithms: bestFit and worstFit [11], which are two topology-agnostic algorithms that are well-known for their simplicity. BestFit starts with a list of open bins, i.e., servers in our case, which is initially empty. When a new VM request arrives, the algorithm finds the most-loaded server that has sufficient capacity to accommodate the VM. If there is no such server, a new empty server is added to the list and the VM is placed into it. If no empty servers exist, it means that there is not enough capacity in the system and the request is denied. Since we simulate VM group requests, we consider that a request is denied when no resources exist for all the VMs of a group cumulatively, i.e., no partial group placements are allowed. WorstFit works similarly, but instead of finding the most-loaded server, it finds the least-loaded server to accommodate every VM. Despite their simplicity, both bestFit and worstFit are extensively utilized in a variety of production systems. For example, the default Kubernetes scheduler is a variant of worstFit, which aims to spread the requests over the computing resources. On the other hand, bestFit (or variants thereof) is a common practice in DCs and is used to minimize the resources fragmentation whenever a request has no specific constraints.

Regarding the system setup, we use the configuration described in section IV. We consider four cases, as far as rack/server level constraints are concerned: Spread/Spread, Spread/Pack, Pack/Pack, and Pack/Spread. Both bestFit and worstFit work at the server level, with tie-breaking as the order of servers. The results of this comparison are shown in Fig. 5.

As depicted, bestFit fills busy servers across all racks, resulting in spreading members of the group across the racks and packing them on servers till full. Thus, bestFit scores better (lower) in the case of Spread over racks (0.23) and Pack over servers (0.19), and worst otherwise (0.77 and 0.81, respectively). Algorithm worstFit does the opposite at the rack level, but achieves better packing on servers since it fills empty servers first. Thus, worstFit scores better (lower) in the case of Pack over racks (0.13) and Pack over servers (0.00), and worst otherwise (0.87 and 1.00, respectively). Our algorithm uses heuristics to satisfy both rack and server level constraints, hence yields almost zero deviation (0.00 to 0.10) at the rack and server levels, except for the case of Pack/Spread at the server level (0.74). The reason for this is that the algorithm packs first at the rack level, then due to (1) the large number of members packed on a rack and (2) the existence of already allocated servers, there is not much room to spread the members across servers in the rack[4]. As shown in the figure, the overall deviation measure is lower for our algorithm, compared to both bestFit and worstFit, in all cases.

VI. Network-Locality Simulation with VM Traces

To study the network-locality properties of *chic-sched*, we assess the performance of an all-level pack configuration and compare it against bestFit. Packing at the lowest levels is of particular importance in oversubscribed networks: it reduces the amount of traffic crossing higher-layer switches, which decreases the likelihood of network congestion and of performance variability due to latency tails for tightly-coupled workloads. Unlike bestFit, *chic-sched* takes topology into account and aims to place the requests in the minimum number of racks possible. The ideal case, where a request fits (and is placed) in a single rack, has two key advantages: (a) communication cost is always one-hop maximum, and (b) network contention due to crossing the network spine is eliminated. In this analysis, we assume that the servers are connected through a single-zone 2-level network as in the example of Fig. 2. As algorithm constraints, we consider (Pack, Hard) for the zone level and (Pack, Soft) for the rack and server levels; the corresponding tuples for the zone and rack/server levels are $[M(n), M(n), M(n)]$, and $[1, M(n), M(n)]$, respectively.

To compare the two algorithms, we extend the CloudSim Plus discrete-event simulator [12], and we use it in conjunction with a set of VM trace files from a production DC. CloudSim Plus is an open-source simulator for modeling of cloud computing infrastructures. For our open-source contributions we refer to [13]. Our system configuration uses 256 racks of 24 servers each. The servers have 48 cores, 384 GiB of memory and 100-Gb/s network interfaces. We extract the VM traces from the publicly-available *AzureTracesForPacking2020* dataset [14], [15], and we use them as an input to CloudSim Plus to simulate the dynamic placement and departure of the VMs. The original dataset covers a 14-day period of >5.5 million VM records, and we focus on three different subsets of group requests in order to study a variety of system loads. For the first subset (*trace A*), we use the first 4K group requests with at least 2 VMs per request, which results in a low-loaded scenario where there are no request denials, i.e., the total system allocation is always <25%. For the request sizes, 56%, 8%, 25%, and 11% of the requests consist of 2, 3, 4, and ≥5 VMs, respectively. For the second subset (*trace B*), we focus on the first 72K group requests with, again, at least 2 VMs per request. In contrast to *trace A*, *trace B* pushes the limits of the core allocation, which results in 18%-26% request denials depending on the placement algorithm, i.e., the system

[4]Extending our algorithm to look ahead over multiple level constraints in order to minimize the deviation measure is future work.

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

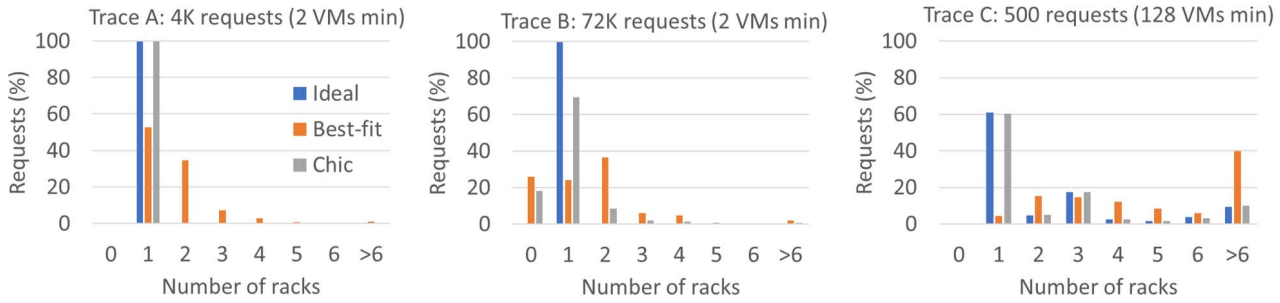


Fig. 6: Placement of VM group requests for three traces from a production Cloud. *Trace A* represents scenarios without request denials and at least 2 VMs per request, *trace B* represents scenarios with 18%-26% request denials and at least 2 VMs per request, while *trace C* represents scenarios without persistent denials and at least 128 VMs per request. The ideal case assumes that every request can be placed in the minimum number of racks, and there are always available resources. *Chic-sched* has better network locality than bestFit for all traces and fewer failures when there are request denials (*trace B*, 0-rack category).

reaches the full capacity. Regarding the request sizes, 64%, 6%, 21%, and 9% of the requests consist of 2, 3, 4, and $\geq$5 VMs, respectively. For the third subset (*trace C*), we use the first 500 group requests with at least 128 VMs per request in order to investigate the performance of the algorithms when only large-scale applications exist in the system. Although the allocation nears system capacity, no persistent saturation is observed like in *trace B*. Regarding the request sizes, 39%, 17%, and 32% of the requests consist of [128, 160], [161, 192], and [193, 224] VMs, respectively, while 12% of the requests consist of $\geq$225 VMs.

Since the dataset does not explicitly identify group requests, we assume that a set of VMs belongs to a VM group request if all the following conditions are met at the same time: (a) the VMs share the same tenant and vmType ids, i.e., they have been requested by the same user account, and they request the same amount of resources, (b) they have the same priority id, i.e., they all have been requested as either high-priority or low-priority, and (c) they start and end at the same times with a maximum difference of 1 sec, which implies that they were possibly requested with the intention to execute a parallel application/workload. The vmTypes are associated with multiple resource request ratios depending on different type of servers (information also anonymized [14]). For the purposes of this analysis, we use the ratios that maximize the requested number of cores. For all traces, the [min / avg / max / stddev] values of the requests are [1s, 1.6d, 90d, 10.5d], [1s, 1.4d, 90d, 9.4d], and [1.2m, 21h, 89d, 5.8d], respectively, while the respective interarrival times of the requests are [0s, 2s, 21s, 3s], [0s, 1s, 31s, 2s], and [0s, 20m, 5.5h, 35m].

Fig. 6 shows the simulation results for both placement algorithms, and it considers also an ideal case, which shows how the requests can be placed by assuming no contention for the resources, i.e., by assuming that (a) every request can be placed in the minimum number of racks, and (b) there are always available resources for the placement. Comparing the ideal case to *chic-sched*, no significant variations are observed for *traces A* and *C*. For *trace B*, around 30.5% fewer requests are placed in a single rack, due to the fact that (i) 18% of the requests are denied since the maximum capacity is reached (0-rack category in Fig. 6), and (ii) 12.5% of the requests are placed in >1 racks due to fragmentation as we approach the maximum capacity. BestFit case has significantly inferior performance in terms of network locality, which is attributed to the topology-agnostic nature of the algorithm. In *trace A*, bestFit only places 53% of the requests in a single rack, while this number is almost 100% for *chic-sched*. The corresponding ratios for *traces B* and *C* are 24% vs 69.5% and 4% vs 60%, respectively. Finally, in *trace B*, where we have request denials, *chic-sched* has less system fragmentation, i.e., 18% denial rate vs 26% for the bestFit case (0-rack category).

VII. Experiments of Placement with Constraints

In this section, we use constraints to place a parallel application within a cluster of a public Cloud, and we measure the performance improvement that can be obtained under a scenario where our heuristics-based scheduler would return a good but not perfect placement. We target workloads sensitive to interference from background network activity or noisy-neighbor VMs. Our application *osnoise* [16] is an open-source benchmark tool to characterize parallel application scaling. It has been developed to represent communication, computation and synchronization characteristics of various real-world tightly-coupled workloads in computational fluid dynamics (CFD), weather forecasting, AI training and other HPC patterns [17]. Our evaluation with this benchmarking tool covers a broad spectrum of tightly-coupled workloads that are sensitive to pack/spread scheduling. *Osnoise* has a calibrated sequence of steps, where each step consists of a compute phase followed by MPI communication. The duration of each compute phase typically ranges between a few to 10s of ms. The communication phase is determined by message size and traffic pattern, and we use a 2D Cartesian grid as our topology. The use of MPI at each step ensures that this benchmark is very similar to many real-world HPC simulations. The benchmark is sensitive to daemon activity or any source of

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

inhomogeneity; if any MPI rank is slowed down at any stage of a step, other MPI ranks will be impacted.

Our application runs on a total of 512 vCPUs, corresponding to 32 worker VMs, each with 16 vCPUs, 64 GB RAM and 25-Gb/s bandwidth. On an additional 24 "noisy-neighbor" VMs of the same profile, we run a bisection bandwidth test that injects network traffic at the maximum supported rate. All VMs run on identical server types. We deploy our application for four placements as shown in Fig. 7 (placements are labeled as P1-P4). For placement P1, we pack the worker VMs on nodes and within the fewest possible racks, and for P2 we add the noise VMs in separate racks. For P3, we place the worker VMs on separate nodes spread over as many racks as possible, while for P4, we add as many noisy-neighbor VMs on the same servers as the workers as possible. We use a total of 19 racks (labeled as rk01-rk19) with up to 4 servers per rack (labeled as s1-s4), and we do not control what third-party workloads run on the servers that we use. As shown in the figure, the placements use the same servers, and only the mix between worker and noise VMs differs between the placements. We note that our placements were generated by requesting 200 VMs in total using the default scheduler and then deleting those VMs that we do not need for our tests; while this approach clearly generates neither perfect pack nor spread placements, it works in our favor as we attempt to characterize performance differences for non-ideal, but good-enough, placements. Using our measure of deviation, we find that for the spread-placement case, 44% of workers have noisy-neighbor VMs on the same server (this is the primary perturbation effect) and 81% of workers of noisy VMs elsewhere in the same rack (a secondary effect).

We define *Effective Parallel Efficiency* (EPE) as *target_Walltime* divided by *elapsed_Walltime*. We measure EPE for (a) three different computation intervals of 1ms (corresponding to a fine-grained parallel workload), 3ms and 10ms, (b) two message sizes of 100B and 500B, and (c) two communication patterns, an all-to-all and a 4-neighbor exchange. The test results for the four placement scenarios are shown in Fig. 8. For three of those scenarios (P1, P2, P3), the measured EPEs are very similar ($\pm 1.5\%$) and range between 62-81%. There is a notable performance degradation of $\sim 9\%$ for the spread+noise scenario P4 that is primarily caused by noisy-neighbor VMs, as described above.

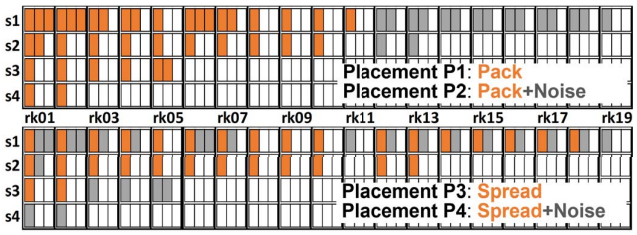


Fig. 7: Four placement experiments (P1-P4) to study noisy-neighbor effects, using 32 worker VMs (orange) and optionally 24 noise VMs (gray). Top: Packing the workers in fewest-possible racks and noise VMs in other racks. Bottom: single worker per node with noise VMs collocated as possible. For all experiments, we use a total of 19 racks (rk01-rk19) with up to 4 servers per rack (s1-s4).

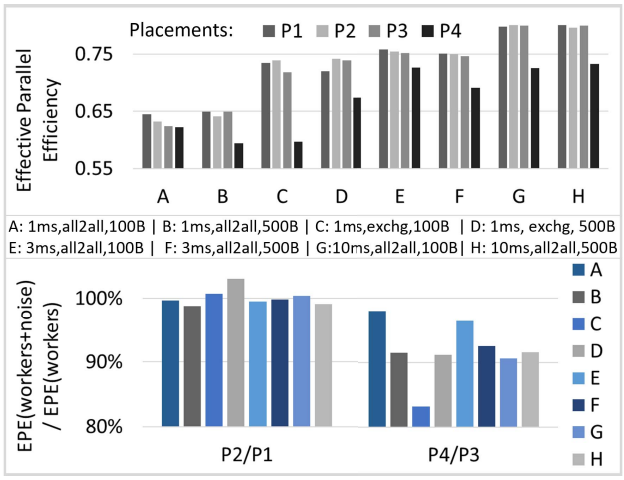


Fig. 8: Top: Effective parallel efficiencies (EPE) measured for the four placements (P1-P4) of Fig. 7. Bottom: Normalized EPE, showing a 9% decrease for the spread placement (P4/P3) as a result of perturbation by (on-node) noisy-neighbors.

VIII. Evaluation of the Algorithm Scalability

In this section, we conduct an experimental evaluation of the scalability of our group placement algorithm. We consider a setup of multiple racks and multiple servers per rack, with a single CPU resource and a background allocation load of 40%, mimicking a bestFit packing policy, as described in section IV. We place a group of a given size with two constraints: (1) soft-spread on racks and (2) soft-pack on servers. We vary the size as follows. Let x be the number of racks, then we set the number of servers per rack to $2x$ and the group size to $8x$. We assume a server capacity of 16 cores and a demand of 2 cores per VM member of the placement group. As described earlier, the background allocation is skewed with probabilities 1/4 and 1/8 for idle and full servers, respectively, and a coefficient of variation of 1.5 for the allocation of the remaining servers.

We simulated eight configurations by varying the value of x from 2 to 64, in powers of 2. For each configuration, we ran 10,000 samples on a laptop (2.6 GHz 6-core Intel Core i7, 16GB RAM). The numerical results in Fig. 9 show the average running times as a function of the DC size and group size. As depicted, a linear regression demonstrates the approximately linear behavior of our placement algorithm. The runtime varied from a fraction of a msec to a maximum of about 8 msec for group of size 512 VMs placed on a system of 8,192 servers in 64 racks. We also ran experiments to explore the effect of background load on the running time of the algorithm. For the three loads we considered—40%, 60%, and 80%—the effect of load on runtime was insignificant ($< 1\%$ deviation from Fig. 9). These experiments demonstrate (1) the linearity of our algorithm in the range of values considered and (2)

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

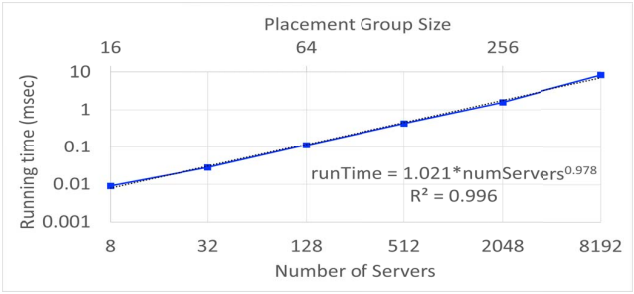


Fig. 9: Scalability of the group placement algorithm.

the invariance to load, and hence the viability of using the algorithm in a scheduler in a real-life sized DC.

We also plot the running time distributions for all traces of section!VI in Fig. 10. The [min/avg/max/stddev] values in ms for *traces A*, *B*, and *C* are [0.2, 0.5, 58.9, 1.2], [0.2, 0.8, 415.8, 4.4], and [1, 8.4, 119.6, 15.1]. *Trace B* has a higher average running times than *trace A*, which is the result of the higher system load. *Trace C* has longer running times than *A* and *B* due to combining far larger group sizes with a high system utilization. Placing big groups in multiple racks requires visiting more tree nodes and performing more sorting operations, which comes at the cost of higher running times. While 10% of running times for groups in *trace C* range between 20-120 ms, this is far faster than for linear schedulers and compares favorably to other heuristics-based schedulers [18]. Overall, the maximum running time of 415.8 ms is observed in the case of *trace B*, which is the scenario with the highest number of denials (18% of the requests are denied).

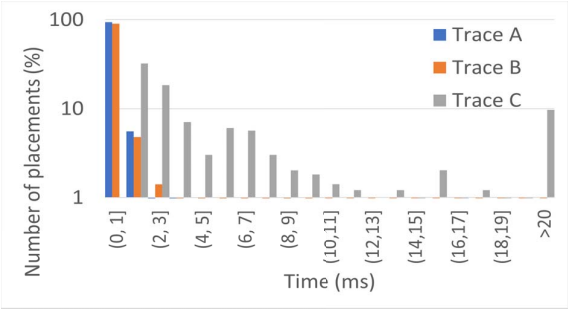


Fig. 10: Distribution of running times for all three traces. *Trace A* represents a case without request denials and at least 2 VMs per request, *trace B* represents a case with 18% request denials and at least 2 VMs per request, while *trace C* represents a case without persistent denials and at least 128 VMs per request.

IX. Related Work

This section presents a high-level comparison between *chic-sched* and a variety of state-of-the-art schedulers, while table III summarizes the most important comparison points.

Major cloud task management frameworks, such as Mesos [19], YARN [20], and Borg [21], include schedulers to place distributed applications onto available resources within a cluster. While these frameworks allow for a wide range of workloads to run on shared infrastructures, their core scheduling algorithms often do not explicitly implement the varied multi-level constraints and topology awareness that we study in this paper. The latest release of YARN comes closest: it includes various types of constraints at the node and rack levels but does not support soft constraints; hence, placement requests are only fulfilled if every single constraint is met, which implies a cost in terms of request failures and system utilization.

TABLE III: Chic-sched vs state-of-the-art schedulers

Scheduler	Pack	Spread	Unidimensional constraints	Multi-level constraints
Mesos [19]	✓	-	-	-
YARN [20]	✓	-	-	-
Borg [21]	✓	✓	-	-
Medea [18]	✓	✓	✓	-
Kubernetes [22]	✓	✓	-	-
Quiet neighborhoods [23]	✓	-	✓	-
Chic-sched	✓	✓	✓	✓

Medea [18], a periodic scheduler for long-running applications, uses a rich set of constraints to place multiple jobs by using an integer programming solver or heuristics. Containers in the applications are tagged and nodes are grouped, resulting in a general graph topology. Our problem is a special case that restricts the topology to a tree, which allows us to optimize constraints at multiple levels independently. As a result, we devise an efficient, heuristics-based algorithm to place groups of any size upon arrival without waiting for other groups.

In Kubernetes [22], the default scheduler does not provide any information about the location of compute nodes, making the cluster appear a flat pool of nodes. KubeFlux [24], a plugin scheduler for Kubernetes, uses a graph-based resource model and graph matching for job allocation. While it provides some topology awareness, it does not have a straightforward specification for placement groups with soft (best-effort) constraints.

Prior work involving topology awareness includes an HPC scheduler that implements the quiet neighborhoods concept [23]. To reduce network sharing and fragmentation, it places jobs within a physical network based on the amount of the requested resources by using left-to-right or right-to-left allocation depending on the job size. While this involves some topology awareness to optimize the resource utilization, requesters do not have the option to influence the placement of their jobs. In contrast, placement of VM groups in a hierarchical DC topology, given an overall objective and additional considerations such as networking, was considered in [25]. However, the resulting constrained optimization problem was solved using the Biased Sampling Algorithm (BSA), rather than heuristics as in this paper.

Finally, recent placement work includes demonstrations for AI-training [26] and inference workloads [27] on GPU clusters, which resemble the Cloud scheduling problem. Gupta et al. [28] demonstrate that topology- and hardware-awareness, cross-VM interference accounting and application-aware consolidation can lead to a 45% improvement in HPC applications performance when scheduled on public clouds.

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.

X. Conclusion

We presented *chic-sched*, a novel placement-group scheduler for distributed workloads on hierarchical topologies with constraints. Our heuristics-based algorithm works without retries and allows for fast scheduling within tens of milliseconds for groups of hundreds of VMs. We introduced a new and simple metric to quantify the goodness of our placements, and we show 20-50% improvement over bestFit or worstFit algorithms for two-level placements with constraints. Our scheduler has better placement locality and fewer failures than bestFit when analyzed with actual Cloud traces. *Chic-sched* may be used as a scheduler plugin within a job management framework, and it enables efficient, fast and quantifiable placement of HPC/AI applications with constraints on shared infrastructures.

XI. Acknowledgment

The information, data, or work presented herein was funded in part by the Advanced Research Projects Agency-Energy (ARPA-E), U.S. Department of Energy, under Award Number DE-AR0000846. The views and opinions of authors expressed herein do not necessarily state or reflect those of the U.S. Government or any agency thereof.

References

[1] D. Reed, D. Gannon, and J. Dongarra, "Reinventing HPC: Challenges and opportunities," *arXiv preprint arXiv:2203.02544*, 2022.

[2] S. Seelam, "Hardware-middleware system co-design for flexible training of foundation models in the cloud," in *Proceedings of the 23rd International Middleware Conference Extended Abstracts*, pp. 2–2, 2022.

[3] "Why we built an AI supercomputer in the Cloud. *Blog post*: https://research.ibm.com/blog/ai-supercomputer-vela-gpu-cluster."

[4] "Alibaba Cloud resource groups, www.alibabacloud.com/help/en/enterprise-distributed-application-service/latest/manage-resource-groups."

[5] "AWS placement groups, https://docs.aws.amazon.com/awsec2/latest/userguide/placement-groups.html."

[6] "Azure proximity placement groups, https://docs.microsoft.com/en-us/azure/virtual-machines/co-location."

[7] "Google Cloud instance placement policies, https://cloud.google.com/compute/docs/instances/define-instance-placement."

[8] "IBM Cloud instance placement groups, https://cloud.ibm.com/docs/vpc?topic=vpc-about-placement-groups-for-vpc."

[9] "Oracle Cloud instance pools, https://docs.oracle.com/en-us/iaas/content/compute/tasks/creatinginstancepool.htm."

[10] B. Shen, R. Sundaram, A. Russell, S. Aiyar, K. Gupta, A. Nagpal, A. Ramesh, and H. Shukla, "High availability for VM placement and a stochastic model for multiple knapsack," in *2017 26th Int. Conf. on Computer Communication and Networks (ICCCN)*, pp. 1–9, 2017.

[11] D. S. Johnson, "Fast algorithms for bin packing," *Journal of Computer and System Sciences*, vol. 8, no. 3, pp. 272–314, 1974.

[12] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, "Cloudsim Plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in *2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)*, pp. 400–406, 2017.

[13] "https://github.com/ppmaniotis/cloudsimplus/releases/tag/v1.0.0."

[14] "AzureTracesForPacking2020, https://github.com/azure/azurepublicdataset/blob/master/azuretracesforpacking2020.md."

[15] O. Hadary, L. Marshall, I. Menache, A. Pan, E. E. Greeff, D. Dion, S. Dorminey, S. Joshi, Y. Chen, M. Russinovich, and T. Moscibroda, "Protean: VM allocation service at scale," in *USENIX Symp. on Operating Systems Design and Impl. (OSDI 20)*, pp. 845–861, 2020.

[16] R. Walkup, "OSNoise tool—utility to characterize parallel application scaling issues, https://github.com/ibm/osnoise."

[17] R. Walkup, S. R. Seelam, and S. Wen, "Best practices for HPC workloads on Public Cloud platforms: A guide for computational scientists to use public cloud for HPC workloads," in *Proceedings of the 2022 ACM/SPEC on ICPE*, pp. 29–35, 2022.

[18] P. Garefalakis, K. Karanasos, P. Pietzuch, A. Suresh, and S. Rao, "MEDEA: scheduling of long running applications in shared production clusters," in *Proc. 13th EuroSys Conference*, pp. 1–13, 2018.

[19] B. Hindman, A. Konwinski, M. Zaharia, A. Ghodsi, A. D. Joseph, R. Katz, S. Shenker, and I. Stoica, "Mesos: A platform for fine-grained resource sharing in the data center," in *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 11)*, 2011.

[20] "YARN - current release, https://hadoop.apache.org/docs/r3.2.3/hadoop-yarn/hadoop-yarn-site/placementconstraints.html."

[21] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. European Conference on Computer Systems*, pp. 1–17, 2015.

[22] Kubernetes, "https://kubernetes.io," 2022.

[23] A. Jokanovic, J. C. Sancho, G. Rodriguez, A. Lucero, C. Minkenberg, and J. Labarta, "Quiet neighborhoods: Key to protect job performance predictability," in *2015 IEEE IPDPS*, pp. 449–459, 2015.

[24] C. Misale, D. J. Milroy, C. E. A. Gutierrez, M. Drocco, S. Herbein, D. H. Ahn, Z. Kaiser, and Y. Park, "Towards standard Kubernetes scheduling interfaces for converged computing," in *Smoky Mountains Computational Sciences and Engineering Conference*, pp. 310–326, Springer, 2021.

[25] A. N. Tantawi, "Quantitative placement of services in hierarchical clouds," in *Quantitative Evaluation of Systems*, pp. 195–210, Springer International Publishing, 2015.

[26] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, *et al.*, "Gandiva: Introspective cluster scheduling for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation*, pp. 595–610, 2018.

[27] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, "MLaaS in the Wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *19th Symposium on Networked Systems Design and Implementation*, 2022.

[28] A. Gupta, L. V. Kalé, D. Milojicic, P. Faraboschi, and S. M. Balle, "HPC-aware VM placement in infrastructure clouds," in *2013 IEEE International Conference on Cloud Engineering*, pp. 11–20, 2013.

Appendix A
Summary of main symbols

TABLE IV: Description of main symbols

symbol	description
$\mathcal{N}$	set of heterogeneous physical entities
$pTree$	physical tree with $\mathcal{N}$ as the set of leaves
$pNode$	node in $pTree$
L	number of levels in $pTree$
$\mathbf{G}$	placement group
$\mathcal{M}$	set of homogeneous logical entities in $\mathbf{G}$
M	group size, $M = \|\mathcal{M}\|$
$\mathcal{C}$	set of level constraints in $\mathbf{G}$
$\mathbf{D}$	resource demand vector of a member of $\mathcal{M}$, with number of resources as length
$\mathbf{A}$	available resource capacity in a physical entity, with number of resources as length
$slot$	a set of resources on a physical entity needed to satisfy the resource demand of a member of $\mathcal{M}$
$lTree$	logical tree
$lNode$	node in $lTree$
node n	a given node in $pTree$
node ν	a child of node n
$A(n)$	number of slots available at the subtree rooted at n
$R(n)$	range at node n, $R(n) = \{a, d, b\}$, where a and b are min and max values, and $d \in [a, b]$ desired value
$M(n)$	number of group members to attempt to place on node n
$N(n)$	number of unvisited sibling nodes of n, including n
$Z(n)$	number of group members placed on node n
$Z^*(n)$	best choice for $Z(n)$
δ	deviation measure, $\delta \in [0, 1]$
$\delta_g(l, n)$	deviation measure for placement group g at level l and size n at level $l + 1$
$\delta_g(l)$	deviation measure for placement group g at level l
δ_g	deviation measure for placement group g

Authorized licensed use limited to: National College of Ireland. Downloaded on October 22,2024 at 14:27:22 UTC from IEEE Xplore. Restrictions apply.